

Slide 1: Understanding Hollow Patterns

What is a Hollow Pattern?

A hollow pattern is a pattern where we print characters only along the **boundary** of a shape.

The inside of the shape is left empty by printing spaces.

For example, a filled square looks like:

```
*****
*****
*****
*****
*****
```

A hollow square looks like:

```
*****
*  *
*  *
*  *
*****
```

The important difference is that the filled square prints * at every position, while the hollow square prints * only at the boundary.

How do we build a hollow square?

Suppose the square has 5 rows and 5 columns.

We can imagine every position as a combination of:

- Row number
- Column number

Column
1 2 3 4 5

```
Row 1 * * * * *
Row 2 *      *
Row 3 *      *
Row 4 *      *
Row 5 * * * * *
```

The stars appear when the position belongs to the border.

What are the border positions?

A position is part of the border when:

- It is in the **first row**
- It is in the **last row**
- It is in the **first column**
- It is in the **last column**

Every other position belongs to the inside.

Key idea

A hollow pattern is not about printing fewer characters randomly. It is about identifying which positions belong to the boundary.

Slide 2: Border Conditions

Now we convert the visual idea into a programming condition.

For every position (`row`, `column`), we ask:

"Is this position on the border?"

If yes:

```
printf("*");
```

Otherwise:

```
printf(" ");
```

Border condition

For a square of size `n`:

```
if (row == 1 || row == n ||  
    column == 1 || column == n)
```

This means:

- `row == 1` → first row
- `row == n` → last row

- `column == 1` → first column
- `column == n` → last column

The `||` operator means **OR**.

So the position needs to satisfy **at least one** of these conditions.

Complete structure

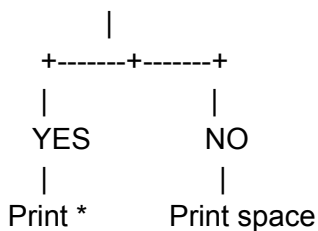
```
for (int row = 1; row <= n; row++)
{
    for (int column = 1; column <= n; column++)
    {
        if (row == 1 || row == n ||
            column == 1 || column == n)
        {
            printf("*");
        }
        else
        {
            printf(" ");
        }
    }

    printf("\n");
}
```

How the program thinks

For every position:

Is this a border position?



Example

For $n = 5$:

```
*****
*   *
*   *
*   *
*****
```

The nested loops still visit **every position**. The `if` statement decides what should be printed there.

Important concept

Loops decide **where to go**.
Conditions decide **what to print**.

That distinction becomes extremely useful for almost every advanced pattern.

Slide 3: Hollow Triangle

The same idea can be applied to a triangle.

Consider this hollow triangle:

```
*
**
* *
* *
*****
```

Look carefully at the pattern.

The stars are not simply based on "print `i` stars."

Instead, we need to identify the **boundary** of each row.

Understanding each row

For 5 rows:

Row	Output	What is happening?
1	*	Only one position
2	**	Both positions are boundaries
3	* *	First and last positions
4	* *	First and last positions
5	*****	Complete bottom boundary

Boundary rule

For a left-aligned hollow triangle, a position should contain * when:

- It is the **first column**
- It is the **last position of the current row**
- It is the **last row**

Conceptually:

if first position

 print *

else if last position

```
    print *  
else if last row  
    print *  
else  
    print space
```

Why do we need the last-row condition?

Look at the bottom:

```
*****
```

The entire bottom row is filled.

Without the last-row condition, we might produce:

```
*  
  
**  
  
* *  
  
* *  
  
*
```

The triangle would not have a proper base.

General lesson

Different shapes can use the same basic programming tools:

Nested loops

+

Row position

+

Column position

+

Condition

=

Pattern

The shape changes, but the programming model remains the same.

Slide 4: X Pattern

The X pattern is different because its characters are positioned along **diagonals**.

Example:

```
*  *
```

```
* *
```

```
 *
```

```
* *
```

```
*  *
```

Imagine the pattern as a grid:

```
* . . *
```

```
. * * .
```

```
. . * .
```

```
. * * .
```

```
* . . *
```

There are two diagonals:

Main diagonal

The first diagonal goes from:

top-left → bottom-right

Its positions look like:

```
* . . . .
. * . . .
. . * . .
. . . * .
. . . . *
```

For a square grid, this happens when:

`row == column`

Opposite diagonal

The second diagonal goes from:

top-right → bottom-left

Its positions look like:

```
. . . . *
. . . * .
. . * . .
. * . . .
* . . . .
```

For a grid of size `n`, this happens when:

`row + column == n + 1`

Combining both diagonals

A position belongs to the X when:

`if (row == column || row + column == n + 1)`

Then:

`if (row == column || row + column == n + 1)`

{


```
    printf("*");  
}  
else  
{  
    printf(" ");  
}
```

The important idea

The program does not know that we are trying to draw an "X."

It simply checks every position:

Is this position on diagonal 1?

OR

Is this position on diagonal 2?

|

YES

↓

Print *

This is a major step in pattern programming:

A visual shape can often be represented as a mathematical condition on row and column positions.

Slide 5: Pattern Conditions

At this point, students should stop thinking of patterns as individual programs.

Instead, they should learn to ask:

Question 1: What are the rows?

For example:

* *

* *

* *

There are 5 rows.

The outer loop controls them:

```
for (int row = 1; row <= 5; row++)
```

Question 2: What are the positions?

Every row contains multiple column positions.

The inner loop controls them:

```
for (int column = 1; column <= 5; column++)
```

So:

Outer loop → rows

Inner loop → columns

Question 3: What should be printed?

At every position, ask:

Should I print a character?

OR

Should I print a space?

For a hollow square:

```

if (row == 1 || row == n ||
    column == 1 || column == n)
{
    printf("**");
}
else
{
    printf(" ");
}

```

For an X:

```

if (row == column ||
    row + column == n + 1)
{
    printf("**");
}
else
{
    printf(" ");
}

```

The **loop structure can remain almost identical**. Only the condition changes.

Pattern-solving framework

Students can use this process for new patterns:

PATTERN



Count the rows



Identify row positions



Identify column positions



Find where characters

should appear



Write the condition



Print character/space



Next row

Three questions to ask for every pattern

Before writing C code, ask:

1. **Where should the character appear?**
2. **Where should the space appear?**
3. **Can I describe the character positions using row and column numbers?**

If the answer to the third question is yes, the pattern can usually be converted into a nested-loop solution.

Chapter 4: Core Takeaway

By the end of this chapter, students should understand these four ideas:

1. Nested loops create the pattern's grid

```
for (row)
{
    for (column)
    {
        ...
    }
}
```

2. Conditions decide what each position contains

```
if (condition)
    printf("***");
else
    printf(" ");
```

3. Different shapes require different position rules

Hollow square:

Border positions

Hollow triangle:

Triangle boundary

X pattern:

Two diagonals

Questions:

1. What is the main idea behind a hollow pattern?

- A) Print * at every position
- B) Print characters only at the boundary and spaces inside
- C) Print spaces only
- D) Print characters only in the first row

 **Hint:** A hollow shape has an empty inside.

 **Glossary:** Boundary → The outer edge of a shape.

 **Answer:** B) Print characters only at the boundary and spaces inside

2. What will be the output of the following C program?

```
#include <stdio.h>
```

```
int main() {
```

```
    int n = 4;
```

```
    for (int row = 1; row <= n; row++) {
```

```
        for (int column = 1; column <= n; column++) {
```

```
            if (row == 1 || row == n ||
```

```
                column == 1 || column == n) {
```

```
                printf("*");
```

```
    }  
    else {  
        printf(" ");  
    }  
}
```

```
printf("\n");  
}
```

```
return 0;  
}
```

A)

B)

*

**

C)

* *

* *

D)

**

**

 **Hint:** A * is printed when the position is in the first row, last row, first column, or last column.

 **Glossary:** Border → The outer boundary of a shape.

 **Answer:** C)

Output:

* *

* *

3. Complete the condition to create a hollow square.

```
for (int row = 1; row <= n; row++) {
```

```
    for (int column = 1; column <= n; column++) {
```

```
        if (row == 1 || row == n ||
```

```
            column == 1 || column == _____) {
```

```
            printf("*");
```



```

    }

    else {

        printf(" ");

    }

}

printf("\n");

}

```

 **Hint:** The last column has the same number as the total number of columns, **n**.

 **Glossary:** Column → A vertical position in the pattern.

 **Answer:** **n**

4. Complete the condition for the main diagonal of an X pattern.

```
for (int row = 1; row <= n; row++) {
```

```
    for (int column = 1; column <= n; column++) {
```

```
        if (row _____ column) {

            printf("*");

        }

        else {

            printf(" ");

        }

    }
}

```

```
printf("\n");  
}
```

 **Hint:** The main diagonal goes from the top-left corner to the bottom-right corner. On this diagonal, the row number and column number are equal.

 **Glossary:** Diagonal → A line of positions moving diagonally across a grid.

 **Answer:** ==

5. Complete the condition to print both diagonals of an X pattern.

```
for (int row = 1; row <= n; row++) {
```

```
    for (int column = 1; column <= n; column++) {
```

```
        if (row == column ||  
            row + column == _____) {
```

```
            printf("*");
```

```
        }
```

```
    else {
```

```
        printf(" ");
```

```
    }
```

```
}
```

```
printf("\n");
```

```
}
```

 **Hint:** For the opposite diagonal, the row and column numbers add up to $n + 1$.

 **Glossary:** $|$ $|$ $\rightarrow$ Logical OR. It means at least one of the conditions must be TRUE.

 **Answer:** $n + 1$